

NTI Workflow

Gendannelse og download fra GitHub

Version 0.7.1

Genereret 2026-06-25

Branch: refactor/app-shell-workflow-split

Release-tag: v0.7.1

Commit: 9e48efb

Indholdsfortegnelse

1. Formål	3
2. Hvad kan gendannes?	3
3. Før lokale mapper slettes	3
4. Installer forudsætninger på en ny pc	4
5. Klon repository igen	4
6. Gendan en bestemt branch	4
7. Gendan en bestemt release	5
8. Genskab Python-miljøet	5
9. Genskab .env	5
10. Start lokalt med Python	6
11. Start med Docker	6
Drift fra registry	6
Lokal build	6
12. Kontrollér at gendannelsen er korrekt	6
13. Hvad GitHub ikke kan redde	7
14. Anbefalet daglig Git-rutine	7
15. Nødsituation: lokal mappe slettet	8
16. Versionsoplysninger	8

1. Formål

Denne vejledning forklarer, hvordan NTI Workflow-projektet gendannes, hvis den lokale projektmappe — for eksempel:

```
# text
C:\GitHub\NTI Workflow
```

— bliver slettet, beskadiget eller flyttet til en ny pc.

ADVARSEL: GitHub er kun en sikkerhedskopi af det, som er committed og pushed. Alt andet skal sikkerhedskopieres separat.

2. Hvad kan gendannes?

Indhold	Kan gendannes fra GitHub	Bemærkning
Committede og pushed filer	Ja	Hentes med <code>git clone</code>
Branches pushed til GitHub	Ja	Kan checkes ud
Tags pushed til GitHub	Ja	Kan checkes ud
Lokale, ikke-pushede commits	Nej	Findes kun lokalt
Ikke-committede ændringer	Nej	Findes kun i working tree
<code>.env</code>	Normalt nej	Er typisk ignoreret af Git
<code>.venv</code>	Nej	Skal genskabes
Lokale Docker-images	Nej	Skal pulls eller bygges igen
Ignorerede ZIP- og analysefiler	Nej	Skal sikkerhedskopieres separat

3. Før lokale mapper slettes

Kør denne kontrol før du sletter noget:

```
# powershell
git status
git branch --show-current
git log -5 --oneline
git remote -v
git tag --list
git push
git push --tags
```

Forklaring:

- `git status` bør være ren (ingen ucommittede ændringer du vil beholde)
- Lokale commits skal pushes til origin
- Tags pushes separat eller med `git push --tags`
- `.env` bør sikkerhedskopieres sikkert (password manager, krypteret backup)
- Vurder untracked filer — de følger ikke med til GitHub

- Commit aldrig secrets, tokens eller kundedata

Kontrol for upushede commits (standard branch er main; tilpas ved behov):

```
# powershell
git status --short
git log origin/main..HEAD --oneline
```

På udviklingsbranch refactor/app-shell-workflow-split:

```
# powershell
git log origin/refactor/app-shell-workflow-split..HEAD --oneline
```

4. Installer forudsætninger på en ny pc

Software	Formål
Git for Windows	Clone, branch, tag
Python 3.11+	Lokal kørsel og tests
Docker Desktop	Valgfrit — containerdrift
Editor (Cursor / VS Code)	Udvikling

Kontrolkommandoer:

```
# powershell
git --version
python --version
docker --version
```

Hvis repository er privat, skal du have adgang til GitHub-kontoen med læserettigheder.

5. Klon repository igen

```
# powershell
New-Item -ItemType Directory -Path C:\GitHub -Force
Set-Location C:\GitHub
git clone https://github.com/andersBpetersen/NTI-Workflow.git "NTI Workflow"
Set-Location "C:\GitHub\NTI Workflow"
```

- `git clone` henter hele Git-historikken
- Standardbranch (main) checkes normalt ud automatisk
- Mapper med mellemrum skal stå i citationstegn

6. Gendan en bestemt branch

List branches:

```
# powershell
git branch -a
```

Skift til udviklingsbranch (eksempel fra aktuel udgivelse):

```
# powershell
git switch refactor/app-shell-workflow-split
```

Hvis branchen kun findes remote:

```
# powershell
git switch --track origin/refactor/app-shell-workflow-split
```

Begreb	Betydning
Lokal branch	Branch på din maskine
Remote branch	Branch på GitHub (origin/...)
Default branch	main (ifølge origin/HEAD)

7. Gendan en bestemt release

```
# powershell
git tag --list
git switch --detach v0.7.1
```

Detached HEAD er velegnet til at inspicere en release, men ændringer bør ikke committes direkte her.

Opret en arbejdsbranch fra tagget:

```
# powershell
git switch -c restore-v0.7.1 v0.7.1
```

Seneste release-tag: v0.7.1

8. Genskab Python-miljøet

```
# powershell
py -m venv .env
.\.env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -r requirements-dev.txt
pytest -q
```

- .env findes ikke på GitHub — opret den på ny
- Aktiveringskommandoen skal køres i hver ny PowerShell-session
- pytest skal køres i det aktive miljø

Hvis PowerShell blokerer scripts:

```
# powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Ændringen gælder kun den aktuelle session.

9. Genskab .env

```
# powershell
Copy-Item .env.example .env
```

- .env.example er en skabelon i repository
- .env kan indeholde lokale værdier og er normalt ikke i GitHub
- Kontrollér image-tag efter gendannelse

Eksempel:

```
# text
NTI_WORKFLOW_IMAGE=tickjf/nti-workflow:0.7.1
```

Kontrol:

```
# powershell
Select-String -Path .env -Pattern "NTI_WORKFLOW_IMAGE"
```

10. Start lokalt med Python

```
# powershell
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app
```

Åbn:

```
# text
http://localhost:8000
```

Kontrollér:

```
# powershell
curl.exe http://localhost:8000/health
curl.exe http://localhost:8000/api/version
```

Forventet version: 0.7.1

11. Start med Docker

Drift fra registry

```
# powershell
Copy-Item .env.example .env
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Kontrollér:

```
# powershell
docker compose -f docker-compose.prod.yml ps
curl.exe http://localhost:8000/health
curl.exe http://localhost:8000/api/version
```

Stop:

```
# powershell
docker compose -f docker-compose.prod.yml down
```

Lokal build

```
# powershell
docker build -t tickjf/nti-workflow:0.7.1 .
```

12. Kontrollér at gendannelsen er korrekt

- ☐ Korrekt branch er aktiv
- ☐ Korrekt tag/version er valgt
- ☐ Working tree er rent
- ☐ Dependencies er installeret

- ☐ Alle tests består
- ☐ .env er genskabt
- ☐ /health svarer 200
- ☐ /api/version viser korrekt version
- ☐ Forsiden åbner
- ☐ Workflow Viewer åbner
- ☐ Vault Config Viewer åbner
- ☐ Docker-image-tag er korrekt

Kommandoer:

```
# powershell
git status --short
git branch --show-current
git describe --tags --always
pytest -q
```

13. Hvad GitHub ikke kan redde

ADVARSEL: Følgende kan normalt ikke hentes fra GitHub:

- Ikke-committede ændringer
- Untracked filer
- Ignorerede filer (.env, .venv, m.m.)
- Lokale adgangstokens og passwords
- Lokale Docker-images
- Virtuelle Python-miljøer
- Filer uden for repository-mappen
- Commits, som aldrig blev pushed

Disse kræver:

- Separat backup eller OneDrive
- Password manager til secrets
- Docker registry (tickjf/nti-workflow) til images
- Regelmæssig git push og git push --tags

14. Anbefalet daglig Git-rutine

```
# powershell
git status
git add .
git commit -m "Beskriv ændringen"
git push
```

For release-tags:

```
# powershell
git tag -a vX.Y.Z -m "Release vX.Y.Z"
git push origin vX.Y.Z
```

ADVARSEL: Kør ikke git add . uden først at kontrollere git status. Commit aldrig .env, tokens eller kundedata. Test før push.

15. Nødsituation: lokal mappe slettet

Hurtig gendannelse på én side:

```
# powershell
New-Item -ItemType Directory -Path C:\GitHub -Force
Set-Location C:\GitHub
git clone https://github.com/andersBpetersen/NTI-Workflow.git "NTI Workflow"
Set-Location "NTI Workflow"

git switch refactor/app-shell-workflow-split

py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -r requirements-dev.txt

Copy-Item .env.example .env
pytest -q
uvicorn app.main:app
```

16. Versionsoplysninger

Felt	Værdi
Aktuel version	0.7.1
Seneste release-tag	v0.7.1
Remote URL	https://github.com/andersBpetersen/NTI-Workflow.git
Dokument genereret	2026-06-25